

Pension Planner Web Application

Functional Specification

Document Version: 9.0

Target Environment: Self-Hosted / Docker

Tech Stack: Spring Boot 3.3+ (Java 25), React (PrimeReact), SQLite

1. System Overview & Architecture Goals

1.1 Purpose

The Pension Planner Web Application enables self-hosted users to track, aggregate, and analyze the performance of their private pensions alongside State Pension forecasts and other retirement income streams to measure progress toward overall retirement goals.

1.2 Target Audience & Deployment Model

- **Self-Hosters & Homelabbers:** Distributed as a single containerized solution containing the Spring Boot backend (serving REST APIs alongside compiled React static assets) and persisting an SQLite database via host volume mount. For optimum performance, consider using Graalvm.

2. Authentication, Authorization & Multi-Tenancy

2.1 Multi-Tenancy Strategy

- **Row-Level Data Isolation:** Every database table includes an explicit foreign key to `userId`.
- **Security Context:** Spring Security enforces filtering by the logged-in user's `userId` across all DB operations to prevent unauthorized access across tenants.

2.2 Registration & User Management

- **Registration Control:** Configurable at runtime via environment variable:
 - `ALLOW_REGISTRATION=true` (Default): Sign-up form accessible via the UI.
 - `ALLOW_REGISTRATION=false`: Rejects `POST /api/v1/auth/register` with 403 Forbidden and hides sign-up option in the UI.
- **Authentication:** Passwords hashed with standard algorithms (e.g., BCrypt/Argon2). Auth state maintained using HTTP-only `SameSite=Lax` JWT cookies, designed for straightforward migration to OIDC in future releases.

3. Database Schema & Audit Specifications

3.1 Core Entity Schema Mapping

Below is the relational schema mapping for SQLite. Every entity includes a mandatory `userId` foreign key.

```

+-----+ +-----+
| Users | | UserSettings |
+-----+ +-----+
| userId (PK, AutoInc) | 1 | id (PK, AutoInc) |
| username (Text, Unique) |-----| userId (FK, Unique) |
| passwordHash (Text) | | targetIncome (Real, Null)|
| createdAt (DateTime) | | retirementDate (DateTime)|
+-----+ +-----+
| | \
| 1 | 1 | \ 1
| | \
| * | * | \ *
+-----+ +-----+ +-----+
| Pension | | StatePension | | otherIncome |
+-----+ +-----+ +-----+
| pensionId (PK, AutoInc) | | id (PK, AutoInc) | | id (PK, AutoInc) |
| userId (FK) | | userId (FK) | | userId (FK) |
| name (Text, Max 100) | | name (Text, Default) | | name (Text) |
| maturityDate (DateTime) | | annualAmount (Real) | | annualAmount (Real) |
| notes (Text, Null) | | notes (Text, Null) | | notes (Text, Null) |
| status (Text/Enum) | +-----+ +-----+
| statusDate (DateTime) |
| color (Text, Null) | (Hex e.g., "#FF5733")
+-----+
| 1
|
| *
+-----+
| PensionStatement |
+-----+
| statementId (PK, AutoInc)|
| pensionId (FK) |
| userId (FK) |
| statementDate (DateTime) |
| planValue (Real) |
| projectedAnnualAmount |
| yearlyCharges (Real,Null)|
| transferValue (Real,Null)|
| amountPaidIn (Real, Null)|
| statementNotes (Text,Null|
+-----+

```

3.2 Audit History Architecture & Schema Specification

To ensure auditability and enable potential historical data rollback/inspection, an automated point-in-time snapshot mechanism is maintained for Pension, PensionStatement, StatePension, and otherIncome.

A. Triggers & Capture Strategy

- **Automated Audit Logging:** Whenever a record in Pension, PensionStatement, StatePension, or otherIncome is created or modified, an exact duplicate snapshot of the entity state prior to mutation (or upon creation) is written to its dedicated Audit History table.
- **Audit Metadata Columns:** Every audit table includes:
 - auditId (Primary Key, Auto-Increment)
 - action (CREATE, UPDATE, DELETE)
 - auditTimestamp (DateTime, Defaults to currentDateAndTime)

B. Audit Schema Definitions

Table: PensionAudit

```
auditId => integer().autoIncrement();
action => text(); // CREATE, UPDATE, DELETE
auditTimestamp => dateTime().withDefault(currentDateAndTime());
pensionId => integer();
userId => integer();
name => text();
maturityDate => dateTime();
notes => text().nullable();
status => text();
statusDate => dateTime();
color => text().nullable();
```

Table: PensionStatementAudit

```
auditId => integer().autoIncrement();
action => text(); // CREATE, UPDATE, DELETE
auditTimestamp => dateTime().withDefault(currentDateAndTime());
statementId => integer();
pensionId => integer();
userId => integer();
statementDate => dateTime();
planValue => real();
projectedAnnualAmount => real();
yearlyCharges => real().nullable();
```

```
transferValue => real().nullable();
amountPaidIn => real().nullable();
statementNotes => text().nullable();
```

Table: StatePensionAudit

```
auditId => integer().autoIncrement();
action => text(); // CREATE, UPDATE, DELETE
auditTimestamp => dateTime().withDefault(currentDateAndTime());
id => integer();
userId => integer();
name => text();
annualAmount => real();
notes => text().nullable();
```

Table: OtherIncomeAudit

```
auditId => integer().autoIncrement();
action => text(); // CREATE, UPDATE, DELETE
auditTimestamp => dateTime().withDefault(currentDateAndTime());
id => integer();
userId => integer();
name => text();
annualAmount => real();
notes => text().nullable();
```

4. UI / React & PrimeReact Frontend Design

The application uses **PrimeReact** components for UI design, data tables, modal dialogs, color picking, date selection, and analytical charting.

4.1 Input & Form Component UX Standards

To maximize usability across different temporal horizons, form inputs—especially date selection—must adhere to the following patterns:

Date Picker Guidelines (PrimeReact Calendar):

- **Long-Range Future Target Dates (UserSettings.retirementDate, Pension.maturityDate):**
 - Configured with view="month", yearNavigator, and dynamic yearRange (e.g., 2026:2080).
 - Avoids day-grid navigation so users can jump directly across decades in 2 clicks without tedious month-by-month paging.
- **Historical & Recent Dates (PensionStatement.statementDate):**
 - Configured with manualInput={true}, monthNavigator, yearNavigator, and showButtonBar={true}.
 - Allows users to directly type dates printed on paper/PDF statements via keyboard or quickly select them via standard calendar view.
- **Current Status Dates (Pension.statusDate):**

- Configured with standard icon picker defaulting to today's date.

4.2 Navigation Routes & Views

1. Dashboard (/)

- **Summary Cards (Card Widgets):**
 - **Total Current Portfolio Value:** Sum of the latest planValue across all active pensions.
 - **Total Projected Annual Income:** Combined total projected annual income (Pensions + State Pension + Other Income) vs. targetIncome from UserSettings.
 - **State Pension & Other Income Overview:** Summary of expected annual payout from State Pension and additional income streams.
 - **Retirement Countdown:** Timeline/countdown widget counting down to the user's retirementDate.
- **Pensions Summary Table (DataTable):**
 - Lists all user pensions (including a color indicator badge/swatch) with inline/row quick actions: **Edit**, **Delete**, and **Add Statement**.
- **Modals (Dialog):**
 - Popups utilizing PrimeReact form components (InputText, Calendar, Dropdown, ColorPicker) for pension creation/editing and statement addition.

2. Pension Details (/pensions/{id})

- **Pension Overview:** Top panel detailing specific pension metadata (Name, Maturity Date, Notes, Status, Status Date, and Assigned Color Swatch).
- **Performance Bar Chart:** PrimeReact Chart component rendering planValue against statementDate grouped by year, styled dynamically using the pension's assigned color.
- **Action Toolbar (Toolbar):** Primary action button to **Add New Statement**.
- **Statements Table (DataTable):**
 - Detailed listing of all statements associated with the selected pension (planValue, projectedAnnualAmount, yearlyCharges, transferValue, amountPaidIn, statementNotes).
 - Quick actions per row: **Edit** statement and **Delete** statement.

3. Other Income (/otherIncome)

- **Performance Bar Chart:** PrimeReact Chart component rendering annualAmount against name.
- **Action Toolbar (Toolbar):** Primary action button to **Add New Other Income**.
- **Other Income Table (DataTable):**
 - Detailed listing of all other income records (name, annualAmount, notes).
 - Quick actions per row: **Edit** and **Delete**.

4. State Pension (/state-pension)

- Form interface to manage state pension details (annualAmount, notes, name).

5. Analytics (/analytics)

Powered by PrimeReact Chart components (Chart.js wrapper) with consistent dynamic color mapping across all views:

- **Retirement Runway / Growth Projection Chart:**
 - Multi-year forward projection chart extending from the current date up to the user's configured retirementDate (from UserSettings).

- Extrapolates portfolio growth using latest statement values (planValue), projectedAnnualAmount trajectories, combined with fixed StatePension and otherIncome streams over time.
- Features a vertical reference line denoting the target retirementDate and milestone markers.
- **Growth vs. Cost (Side-by-Side Comparison):**
 - Grouped Bar/Line Chart allowing side-by-side comparison per pension.
 - Metrics: Valuation Growth (planValue - amountPaidIn) vs. Cumulative Charges (yearlyCharges) per pension.
- **Combined Growth vs. Target (Stacked Bar Chart):**
 - **Stacked Bar Visualization:** Aggregates yearly values of all pensions (each pension rendered as a colored segment using its database-stored color), plus the expected StatePension and stacked otherIncome annual amounts.
 - **Target Goal Overlay:** A horizontal benchmark target line representing targetIncome (from UserSettings) overlaying the stacked bars to evaluate progress toward overall retirement income targets.
- **Historical Portfolio Trend:** Multi-line chart tracking individual and total planValue over past statement dates.

6. Settings (/settings)

- User profile configuration form to adjust targetIncome and retirementDate (UserSettings).

5. Non-Functional & Operations Specifications

5.1 Technology Stack Summary

- **Language/Runtime:** Java 25.
- **Backend Framework:** Spring Boot 3.3+ (Spring Web, Spring Security, Spring Data JPA / JDBC, Spring Entity Listeners for Audit Triggers).
- **Frontend Framework:** React with PrimeReact Component Library (including ColorPicker, Calendar, and Chart), PrimeFlex grid, and PrimeIcons.
- **Database Engine:** Embedded SQLite driver (e.g., sqlite-jdbc) mounted to a host path (e.g., /data/pension_planner.db).

5.2 Environment Variables

Variable	Description	Default
SERVER_PORT	HTTP port for the combined Spring Boot service	8080
DATABASE_PATH	Host file path for SQLite database storage	/data/pension.db

ALLOW_REGISTRATION	Toggle public sign-up functionality	true
JWT_SECRET	Secret key for JWT signature validation	Auto-generated